PortBlast

Operator Manual & Command Reference

A scope-gated wrapper that chains RustScan, Nmap, SearchSploit,
Hydra, John the Ripper, and Metasploit into one authorized-assessment workflow.

Version 0.1.0 • CyberSecurity_Lab

Authorized use only

Authorized Use & Rules of Engagement

> **Run this only against systems and networks you own or are explicitly authorized to test.**
> PortBlast drives **real** offensive tooling (port scanning, service enumeration, credential testing, and exploitation) against **real** hosts. Running it against systems you do not own or have written permission to test may be illegal and can cause service disruption. You are solely responsible for staying within authorized scope.

Unlike the loopback-only simulation labs elsewhere in this repository, PortBlast is not a sandbox. Its entire safety model is an **explicit scope allowlist**:

- Every network-facing command validates its target against `scope.json` **before a single packet is sent**. A target that is not a literal IP address inside an authorized network is refused.
- The shipped default scope is private lab ranges only (`192.168.1.0/24` and `10.10.10.0/24`). Edit the scope file to match your engagement — and keep it narrow.
- The `crack` command is the one exception: it reads a local hash file and performs no network I/O, so it is not scope-gated.

Before each engagement: confirm written authorization, confirm the target IPs fall inside your signed scope, and set `scope.json` to match exactly. When in doubt, stop.

1 What PortBlast Is

PortBlast is a small Python command-line tool that orchestrates six industry-standard security binaries into a single, repeatable assessment workflow. It does not reimplement any of them — it detects what you have installed, calls each tool with safe (list-based, non-shell) arguments, parses the output into normalized data, and writes a timestamped JSON report.

The workflow it models:

- **Discover** — fast TCP port discovery with RustScan.
- **Enumerate** — service and version detection with Nmap.
- **Research** — passive Exploit-DB lookup of each service with SearchSploit.
- **Test credentials** — online password testing with Hydra; offline hash cracking with John the Ripper.
- **Exploit** — single, operator-chosen Metasploit module execution.

> **PortBlast never auto-exploits.**
> The `scan` stage only *reports* candidate exploits. Running a module is always a separate, deliberate `exploit` command where you choose the module and its options.

2 Prerequisites

2.1 Python

Python **3.9 or newer**. PortBlast uses only the Python standard library — there is nothing to `pip install` for the tool itself. Confirm your version:

```
py --version        # Windows
python3 --version   # Linux / macOS
```

2.2 External tools

PortBlast **detects but does not install** the underlying binaries. Install the ones you need and make sure each is on your PATH. Each command only requires the tools it actually uses, so you can start with just Nmap.

Tool	Used by	Required?
RustScan	scan (discovery)	Optional — falls back to a full Nmap scan
Nmap	scan (enumeration)	Required for scan
SearchSploit (exploitdb)	scan (passive lookup)	Optional
Hydra (thc-hydra)	bruteforce	Required for that command
John the Ripper	crack	Required for that command
Metasploit (msfconsole)	exploit	Required for that command

2.3 Installing the tools

The fastest route is a security distribution such as Kali Linux or Parrot OS, where every tool above is preinstalled. Otherwise install per platform:

Kali / Parrot (preinstalled, or):

```
sudo apt update
sudo apt install nmap hydra john exploitdb metasploit-framework
cargo install rustscan   # or grab a release binary
```

macOS (Homebrew):

```
brew install nmap hydra john-jumbo rustscan
brew install --cask metasploit
```

Windows: Nmap has an official installer (nmap.org). RustScan ships Windows release binaries. Hydra, John, SearchSploit, and Metasploit are smoothest under WSL2 (install the Linux packages inside your WSL distro). Run `py portblast.py tools` to confirm what PortBlast can see on your PATH.

2.4 Getting PortBlast

```
cd CyberSecurity_Lab/PortBlast
py portblast.py tools        # verify environment
```

All commands are run from inside the `PortBlast/` directory, because the modules import each other by name.

3 Configuring Scope

Scope lives in `scope.json`, next to the scripts. It is the first thing to set for a new engagement and the single control that keeps PortBlast inside authorized ground.

```
{
    "authorized_networks": [
        "192.168.1.0/24",
        "10.10.10.0/24"
    ]
}
```

- Entries may be any network that Python's `ipaddress` module understands — IPv4 or IPv6, CIDR or a single host (e.g. `192.168.1.50/32`).
- A target must be a **literal IP address** inside one of these networks. Hostnames are refused by design (no surprise DNS resolution to an out-of-scope address).
- An empty or missing list is a hard error — PortBlast will not run any network command with no scope defined.

> **Keep scope as tight as the engagement allows.**
> A `/32` per authorized host is safer than a whole `/24`. Widen it only when your authorization explicitly covers the broader range.

4 Command Reference

General form:

```
py portblast.py <command> [arguments] [options]
```

Global options: `--version` prints the version; `--help` (or no command) prints usage. Most commands accept `--silent` to suppress console output. On any scope or tool error, PortBlast prints `[!] <reason>` to stderr and exits with status 1.

Command	Purpose	Scope-gated?
tools	List which security tools are installed	n/a
scan	Discover ports, enumerate services, research exploits	Yes
crack	Crack a local hash file (John the Ripper)	No (local only)
bruteforce	Online credential testing of one service (Hydra)	Yes
exploit	Run one chosen Metasploit module	Yes

4.1 tools — environment check

Prints the platform, Python version, and a FOUND / NOT FOUND line for each supported binary. Run this first on any new machine.

```
py portblast.py tools
```

4.2 scan — discover + enumerate + research

`py portblast.py scan <target-ip> [--silent]`

Pipeline: RustScan discovers open TCP ports (if installed); Nmap `-sV` then fingerprints services/versions on those ports (or runs a full scan if RustScan is absent or found nothing); when SearchSploit is present, each product/version is looked up in the local Exploit-DB and any matches are listed inline as `! EDB-<id>`.

Output: an OPEN SERVICES table to the console, plus a full JSON report at `reports/<ip>_<UTC-timestamp>.json`. `--silent` suppresses the console table; the report is still written.

Examples:

```
py portblast.py scan 192.168.1.50
py portblast.py scan 10.10.10.20 --silent
```

4.3 crack — John the Ripper (local, not scope-gated)

`py portblast.py crack <hash_file> [--format FMT] [--wordlist FILE] [--silent]`

Runs a cracking pass over the hash file, then `john --show`, and prints the recovered `username password` pairs. Operates purely on a local file.

Option	Meaning
hash_file	Path to the hash file to crack (required).
--format FMT	John hash format, e.g. raw-md5, nt, sha512crypt.
--wordlist FILE	Wordlist to use, e.g. rockyou.txt.
--silent	Suppress console output.

```
py portblast.py crack hashes.txt --format raw-md5 --wordlist rockyou.txt
```

4.4 bruteforce — Hydra (scope-gated)

`py portblast.py bruteforce <target-ip> <service> (-l USER | -L FILE) (-p PASS | -P FILE) [--silent]`

Tests credentials against a single service on an authorized host and prints any recovered logins. You must supply at least one username source and one password source.

Argument / flag	Meaning
target	Authorized target IP (validated against scope).
service	Hydra service module: ssh, ftp, http-get, smb, rdp, …
-l / --login	A single username.
-L / --login-list	A file of usernames (one per line).
-p / --password	A single password.
-P / --password-list	A file of passwords (one per line).
--silent	Suppress console output.

```
py portblast.py bruteforce 192.168.1.50 ssh -l root -P passwords.txt
py portblast.py bruteforce 10.10.10.20 ftp -L users.txt -P rockyou.txt
```

Online brute forcing is loud and can lock accounts.
It generates real authentication traffic against the target. Use small, targeted wordlists, and only where your authorization covers credential testing.

4.5 exploit — Metasploit (scope-gated)

`py portblast.py exploit <target-ip> --module <path> [--option KEY=VALUE ...] [--silent]`

Builds a one-shot Metasploit resource script — `use <module>` → `set RHOSTS <target>` → each `--option` → `run` → `exit` — and runs it under `msfconsole -q`. The raw console output is printed. **You** choose the module and options; PortBlast does not pick exploits for you.

Argument / flag	Meaning
target	Authorized target IP (validated against scope; becomes RHOSTS).
--module	Metasploit module path (required), e.g. exploit/multi/handler.
--option KEY=VALUE	A module option. Repeatable; value may itself contain '='.
--silent	Suppress PortBlast's own banner output.

```
py portblast.py exploit 192.168.1.50 \
```

```
    --module exploit/multi/handler \
    --option LPORT=4444 --option PAYLOAD=windows/meterpreter/reverse_tcp
```

5 Reports

Every `scan` writes a JSON report to the `reports/` directory (created automatically, and git-ignored). Filenames are `<ip>_<UTC-timestamp>.json`, so repeated scans never overwrite each other. Structure:

```
{
    "tool": "PortBlast",
    "version": "0.1.0",
    "timestamp": "2026-10-11T18:04:22.511+00:00",
    "target": {
        "address": "192.168.1.50",
        "hostname": "lab.local",
        "services": [
            {
                "port": 22, "protocol": "tcp", "state": "open",
                "name": "ssh", "product": "OpenSSH",
                "version": "8.9", "extra_info": "Ubuntu",
                "exploits": [
                    {"title": "...", "edb_id": "50000", "path": "..."}
                ]
            }
        ]
    }
}
```

6 Running the Tests

PortBlast ships a unit-test suite that **mocks out every external binary**, so it runs correctly with none of the tools installed. Use it to confirm the parsers and scope logic after any change.

```
py test_portblast.py
```

The suite covers scope validation, the RustScan and Nmap-XML parsers, the SearchSploit JSON parser, the Hydra output regex, the John `--show` filter, the Metasploit resource-script builder, and the key=value option parser (22 tests).

7 Troubleshooting

Symptom	Cause & fix
[!] Target … is outside the authorized PortBlast scope	The IP is not inside any network in scope.json. Add the authorized network, or confirm you are testing the right host.

Symptom	Cause & fix
PortBlast requires an IP address as the target	You passed a hostname. Resolve it yourself and pass the literal IP (prevents accidental out-of-scope DNS).
Executable not found: <tool>	The binary is not on PATH. Install it, or run py portblast.py tools to see what is detected.
Nmap is required for service enumeration	scan needs Nmap even when RustScan is present. Install Nmap.
No authorized networks configured	scope.json has an empty or missing authorized_networks list. Add at least one network.
Invalid scope configuration	scope.json is not valid JSON. Check commas and brackets.

8 Project Layout

File	Responsibility
portblast.py	CLI entry point and command dispatch.
scope.py	Scope allowlist loading and target validation.
scanner.py	RustScan discovery + Nmap enumeration / XML parsing.
vuln.py	SearchSploit (Exploit-DB) lookup.
creds.py	Hydra credential testing + John hash cracking.
exploit.py	Metasploit module execution.
reporting.py	JSON report writer.
models.py	Service / Target dataclasses.
tools.py	Tool + platform detection.
scope.json	The authorized-network allowlist.
test_portblast.py	Unit tests (all external tools mocked).